

Documentação dos Diagramas C4 — Sistema WorkFlow FATEC

Este documento explica os 4 diagramas do arquivo `C4_WorkFlow_FATEC.drawio`, descrevendo os elementos representados e os processos do sistema em cada nível de zoom, conforme o Modelo C4 (Simon Brown).

O Modelo C4 organiza a arquitetura em quatro níveis, cada um "ampliando" para dentro do anterior:

Nível	Diagrama	Pergunta que responde
1	Contexto do Sistema	Quem usa o sistema e com que outros sistemas ele se comunica?
2	Container	Quais são as grandes peças executáveis/implantáveis do sistema?
3	Componente	Como uma dessas peças é organizada internamente?
4	Dinâmico	Como essas peças colaboram para executar um processo específico?

1. Diagrama de Contexto do Sistema (System Context)

Objetivo: dar a visão mais ampla possível — o sistema como uma caixa única, cercado pelas pessoas que o usam e pelos sistemas externos com que ele troca informação. Não mostra nenhum detalhe técnico interno.

Elementos:

- **Aluno** (pessoa) — usuário que abre e acompanha solicitações.
- **Secretaria / Backoffice** (pessoa) — usuária que faz a gestão administrativa do sistema.
- **Setor Responsável** (pessoa) — usuário do setor para o qual um chamado é encaminhado (ex.: financeiro, coordenação, TI acadêmica).
- **Sistema WorkFlow FATEC** (sistema, caixa central) — o sistema em si.
- **AWS SES** (sistema externo) — serviço de e-mail usado para comunicações automáticas.

Processos representados:

- *Aluno* → *Sistema*: o aluno abre um chamado (categoria, serviço, dados, anexos) e acompanha seu andamento até a resolução.
- *Secretaria/Backoffice* → *Sistema*: faz a triagem inicial dos chamados, gerencia usuários, categorias e serviços do catálogo.
- *Setor Responsável* → *Sistema*: recebe os chamados encaminhados à sua área, analisa a demanda e registra a resolução.
- *Sistema* → *AWS SES*: sempre que é necessário notificar por e-mail (ex.: redefinição de senha, primeiro acesso), o sistema dispara o envio via SES.

Este diagrama é o ponto de partida para qualquer pessoa nova no projeto entender "o que o sistema faz e para quem", sem entrar em stack tecnológica.

2. Diagrama de Container

Objetivo: abrir a caixa "Sistema WorkFlow FATEC" e mostrar suas grandes unidades executáveis — aquilo que roda ou é implantado separadamente (uma aplicação web, uma API, um banco de dados etc.).

Elementos dentro da fronteira do sistema:

- **Aplicação Web (Frontend)** — Next.js/React. É por onde aluno, secretaria e setor interagem. O `middleware.ts` do projeto protege as rotas `/admin` e `/aluno`, validando a sessão antes de liberar o acesso.
- **API Application (Backend)** — Node.js/Express. Concentra toda a lógica de negócio: autenticação, chamados, catálogo, notificações e analytics, expostos como API REST/JSON.
- **Banco de Dados** — MySQL 8, acessado via Prisma ORM. Armazena usuários, chamados, categorias/serviços, notificações e o histórico de status de cada chamado.
- **Adminer** (*tracejado, ambiente de dev*) — interface web para inspecionar/administrar o banco diretamente, usada só em desenvolvimento — por isso não faz parte do "sistema em produção" propriamente dito.

Fora da fronteira:

- Os três atores (Aluno, Secretaria/Backoffice, Setor Responsável), que acessam tudo pela Aplicação Web.
- **AWS SES**, acionado pela API para envio de e-mails.
- Uma nota separada para o **pipeline de CI/CD** (GitHub Actions → ECS Fargate), que não é um container em tempo de execução, mas é relevante para entender como o sistema chega à produção.

Processo representado:

1. O usuário (qualquer um dos três papéis) acessa a Aplicação Web via HTTPS.
2. A Aplicação Web consome a API do Backend via chamadas JSON/HTTPS (fetch).

3. O Backend lê e grava dados no MySQL através do Prisma.
4. Quando necessário, o Backend aciona o AWS SES para enviar e-mails.
5. Em desenvolvimento, o Adminer acessa o banco diretamente para fins de depuração/administração.

Esse é o diagrama mais útil para decisões de infraestrutura e deploy — mostra exatamente o que precisa estar no ar (frontend, backend e banco) e o que é só apoio (Adminer, CI/CD).

3. Diagrama de Componentes

Objetivo: abrir o container **API Application** e mostrar como ele é organizado internamente em módulos de responsabilidade única.

Componentes:

- **Auth Middleware** — intercepta as requisições, valida o JWT e as permissões do usuário; também é responsável por revogar sessões quando a senha é redefinida.
- **Auth Component** — cuida do login, emissão/renovação (refresh) do token JWT e do fluxo de troca obrigatória de senha.
- **Catalog Component** — gerencia as categorias e os serviços disponíveis no catálogo de solicitações.
- **Tickets Component** — o "coração" do sistema: cria chamados, valida a categoria/serviço escolhido e controla as transições de status (ex.: `AGUARDANDO_USUARIO`, `RESOLVIDO_ENCERRADO`).
- **Notifications Component** — publica notificações in-app, respeitando a preferência de notificação de cada usuário/setor.
- **Analytics Component** — agrega estatísticas de chamados em tempo real, usado pelo endpoint `/tickets/stats`.
- **Email Adapter** — camada de integração que efetivamente conversa com o AWS SES.
- **Data Access (Prisma Client)** — camada única de acesso a dados, usada por todos os outros componentes para falar com o banco.

Processos representados:

- Toda requisição autenticada passa primeiro pelo **Auth Middleware**, que autoriza (ou barra) o acesso aos componentes de Tickets, Catalog, Notifications e Analytics.
- O **Tickets Component** depende do **Catalog Component** para validar a categoria/serviço escolhido, e aciona o **Notifications Component** sempre que cria ou atualiza o status de um chamado.
- O **Auth Component** aciona o **Email Adapter** para os e-mails de redefinição de senha/primeiro acesso, que por sua vez fala com o AWS SES.
- Todos os componentes de negócio (Tickets, Catalog, Notifications, Analytics) persistem e consultam dados exclusivamente através do **Data Access**, que centraliza o acesso ao MySQL.

Esse diagrama é o mais próximo do código — ajuda diretamente quem vai mexer no backend a entender onde cada responsabilidade mora.

4. Diagrama Dinâmico — Fluxo de Abertura de Chamado

Objetivo: diferente dos três anteriores (que são "estáticos"), este mostra a colaboração entre elementos ao longo do tempo, para **um processo específico** — neste caso, o processo de abertura de um chamado, espelhando o seu diagrama BPMN.

Sequência numerada:

1. **Aluno → Aplicação Web:** preenche categoria, serviço e os dados do chamado.
2. **Aplicação Web → Auth Middleware:** envia **POST /tickets** com o JWT do aluno.
3. **Auth Middleware → Tickets Component:** confirma que o token e as permissões são válidos, e libera a requisição.
4. **Tickets Component → Catalog Component:** valida se a categoria/serviço escolhido existe e está ativo.
5. **Tickets Component → Data Access:** grava o novo chamado com o status inicial.
6. **Data Access → Banco de Dados:** executa o **INSERT** do chamado.
7. **Tickets Component → Notifications Component:** publica a notificação tanto para o setor responsável quanto para o próprio aluno.
8. **Notifications Component → Data Access:** grava a notificação in-app (respeitando a preferência de cada destinatário).
9. **Aplicação Web → Aluno:** confirma na tela que a solicitação foi aberta com sucesso.
10. **Setor Responsável → Aplicação Web:** acessa o chamado recém-criado para iniciar a triagem.

Essa sequência corresponde diretamente às raias **USUÁRIO → SISTEMA → SETOR RESPONSÁVEL** do seu BPMN: os passos 1–2 são a raia do usuário, 3–8 são a raia do sistema (validação, criação do chamado, notificação), e o passo 10 já inicia a raia do setor responsável (triagem/análise da demanda).

Como os diagramas se conectam

- O **Contexto** diz *quem* participa.
- O **Container** diz *em que pedaço de software* cada interação acontece.
- O **Componente** diz *qual módulo dentro do backend* trata cada responsabilidade.
- O **Dinâmico** amarra tudo isso em um processo de ponta a ponta, ligando o fluxo de negócio (BPMN) à arquitetura técnica.

Se quiser, posso criar diagramas dinâmicos adicionais para outros processos do BPMN (ex.: "complementação de dados pelo aluno" ou "resolução do chamado pelo setor"), seguindo o mesmo padrão.